

AI Care Navigator

Insurance-Aware Hospital & Treatment Navigation Platform

Round 2 — Production-Level Project Submission

Project Documentation

Team rock

Tech stack: Next.js · Express.js · MongoDB · JWT Auth · Docker

[Live App](#) · [API](#) · [Source Code](#)

[Demo vidio](#)

1. Executive Summary

AI Care Navigator is a full-stack, insurance-aware decision-support platform built for the Precision Care Challenge 2026 (Round 2 — Production-Level Project Submission). It helps patients and caregivers navigate hospital admission and treatment by mapping their insurance coverage — government schemes such as ESI, PM-JAY, Arogya Karnataka, and Yeshaswini, as well as private or employer insurance — to eligible hospitals, room categories, and care pathways.

The platform consolidates information that is normally scattered across policy documents, hospital portals, and manual processes into a single, structured, easy-to-use interface. It tracks a patient's care journey stage by stage (Pre-Admission through Discharge) and surfaces contextual, insurance-aware guidance at each step — entirely as decision support, never as medical diagnosis or binding insurance advice.

The system is a genuine, working production-grade application: a Next.js frontend, an Express.js REST API backend, and a MongoDB database, containerized with Docker and deployed live on Vercel and Render. The only synthetic component is the hospital dataset, which follows the challenge's own data guidance to use synthetic or mock data in place of real hospital/insurer data feeds.

2. Problem Statement

A large majority of individuals in India are covered by one or more health insurance schemes — private, employer-provided, or government schemes such as ESI, PM-JAY, Arogya Karnataka, and Yeshaswini. During medical emergencies or hospital admissions, patients and caregivers frequently face confusion due to a lack of timely, structured information:

- Which hospitals are covered under a given policy or scheme
- What type of room is eligible under the coverage
- What procedures or consumables are reimbursable
- What alternatives exist if the first choice is not viable

These questions are frequently answered too late — after admission, or even after discharge — resulting in financial stress, delayed care, or suboptimal decisions. Insurance policy details, hospital capabilities, and treatment progress information exist across disconnected documents, portals, and manual processes, and this information is rarely available to caregivers in a form they can act on during a crisis.

3. Proposed Solution

AI Care Navigator unifies insurance information, hospital data, and treatment journey tracking into one platform that helps users navigate hospital care using their insurance coverage as a guiding constraint. The platform:

- Maps a user's insurance details to hospital options and treatment pathways
- Suggests eligible hospitals, room categories, and care alternatives based on available data, with a plain-language explanation for every suggestion
- Tracks the patient's treatment journey and continuously surfaces relevant insurance-aware guidance at each stage

The solution functions strictly as a decision-support and information platform. It does not provide medical diagnoses, clinical treatment recommendations, or binding insurance advice — every response in the application carries this disclaimer explicitly.

4. System Architecture

The application follows a standard three-tier architecture, fully containerized for local development and deployed as independent services in production.

Tier	Technology	Responsibility
Frontend	Next.js 14 (App Router), React, Tailwind CSS	Landing page, authentication, dashboard, insurance form, hospital search, care-journey timeline
Backend	Node.js, Express.js, JWT, bcrypt	REST API, authentication, matching engine, guidance engine
Database	MongoDB + Mongoose	Users, insurance policies, hospitals, care journeys

Request flow: the browser talks only to the Next.js frontend, which calls the Express API over HTTPS with a JWT bearer token for authenticated routes. The API applies business logic (eligibility matching, stage guidance) and reads/writes MongoDB. The frontend never talks to the database directly.

4.1 Local development (Docker)

A docker-compose.yml spins up three containers — mongo, backend, and frontend — connected on an internal Docker network, enabling one-command local setup (docker-compose up --build).

4.2 Production deployment

Component	Platform	Notes
Frontend	Vercel	Deployed directly from the frontend/ directory; builds natively (Docker not used for this step)
Backend	Render (Web Service, Docker environment)	Built from backend/Dockerfile; environment variables configured in the Render dashboard
Database	MongoDB Atlas (M0 free cluster)	Managed, cloud-hosted MongoDB instance

5. Core Features

5.1 Insurance & User Data Ingestion

Users enter structured insurance details — scheme type, insurer name, policy type, coverage limit, room eligibility, exclusions, co-pay percentage, and sub-limits. This data is normalized into a standard internal schema (the InsurancePolicy model) regardless of whether the source is a government scheme or a private insurer.

5.2 Hospital & Care Option Mapping

Given a saved insurance policy and an optional city/specialty filter, the matching engine (`matchEngine.js`) evaluates every hospital in the dataset and returns only those that are in-network for the policy, with:

- The room categories the policy makes the user eligible for at that hospital
- Indicative per-day cost for each eligible room category
- A ranked score based on specialty match, city match, and hospital rating
- A plain-language explanation of why each hospital was suggested

5.3 Care Journey Tracking & Contextual Suggestions

A care journey progresses through six stages: Pre-Admission, Admission, Investigation, Procedure, Recovery, and Discharge. At every stage, the guidance engine (`guidanceEngine.js`) generates context-aware, insurance-specific prompts — for example, warning that booking a room above the eligible category can lead to a proportionate deduction on the entire claim, or reminding the user to retain bills for reimbursement at discharge.

5.4 User-Facing Experience

A simple, responsive web interface displays the insurance summary and key constraints, ranked hospital suggestions with cost and eligibility, and a stage-stepper timeline of the care journey with guidance surfaced at each step — designed for usability under stressful conditions.

5.5 Non-Diagnostic Guardrail

Every suggestion and guidance response in the application includes an explicit disclaimer: this is decision-support information only, not a medical diagnosis or a binding insurance approval, and the user should always confirm final eligibility with their insurer and hospital.

6. Technology Stack

Layer	Technologies
Frontend	Next.js 14 (App Router), React 18, Tailwind CSS
Backend	Node.js, Express.js 4, express-validator
Database	MongoDB, Mongoose ODM
Authentication	JSON Web Tokens (JWT), bcrypt password hashing
DevOps	Docker, Docker Compose, multi-stage Dockerfiles
Hosting	Vercel (frontend), Render (backend), MongoDB Atlas (database)

7. Database Schema

7.1 User

- name, email (unique), password (bcrypt-hashed), role (patient / caregiver), phone

7.2 InsurancePolicy

- user (reference), schemeType (ESI / PM-JAY / Arogya-Karnataka / Yeshaswini / Private / Employer / Other)
- insurerName, policyNumber, policyType, coverageLimit, remainingCoverage
- roomEligibility, networkType, exclusions[], coveredProcedures[], copayPercentage, subLimits[]

7.3 Hospital

- name, city, state, address, specialties[]
- networkSchemes[], networkInsurers[], roomTypes[] (category, indicative cost/day, availability)
- indicativeProcedureCosts[], rating, beds, contact

7.4 CareJourney

- user, insurancePolicy, hospital (references), patientName, selectedRoomCategory
- currentStage, stageHistory[] (stage, timestamp, notes), notes, closed

8. API Documentation

Method	Endpoint	Auth	Description
POST	/api/auth/register	—	Create a new user account
POST	/api/auth/login	—	Authenticate and receive a JWT
GET	/api/auth/me	Yes	Get the current authenticated user
POST	/api/insurance	Yes	Add a new insurance policy
GET	/api/insurance	Yes	List the current user's policies
PUT	/api/insurance/:id	Yes	Update a policy
DELETE	/api/insurance/:id	Yes	Delete a policy
GET	/api/hospitals	—	Browse the hospital dataset
POST	/api/hospitals/match	Yes	Ranked, explained hospital/room matches for a policy
POST	/api/journeys	Yes	Start a new care journey
GET	/api/journeys	Yes	List the user's care journeys
GET	/api/journeys/:id	Yes	Get a journey with current-stage guidance

Method	Endpoint	Auth	Description
PATCH	/api/journeys/:id/stage	Yes	Advance the journey's stage; returns fresh guidance

9. Note on Mock / Synthetic Data

Per the challenge's own data guidance — “No data will be provided from GE HealthCare and will rely on participants' enterprise to obtain relevant information” and the explicit instruction to “Use only synthetic or user-provided mock data for insurance details” — the hospital dataset in this submission (10 representative hospitals across Bengaluru, Mysuru, Mumbai, Pune, Delhi, Hyderabad, and Chennai, covering ESI, PM-JAY, Arogya Karnataka, Yeshaswini, and private insurer networks) is synthetic and hand-authored for demonstration.

Everything else in the application is genuinely functional, not simulated: user accounts, insurance policies, hospital matches, and care journeys are all real database records created through real API calls, exactly as they would be in production. The only change required to move from this MVP to a real-world deployment is replacing the contents of one file — `seed/data/hospitals.json` — with a real, licensed hospital/insurer dataset (for example, National Health Authority PM-JAY empanelled hospital data, or direct integrations with state scheme portals and private insurers).

10. Roadmap to Real-World Deployment

Area	Planned Change
Hospital data	Replace synthetic dataset with real PM-JAY / state-scheme / insurer network data
Insurance verification	Insurer API integration to verify policy numbers; document upload with OCR auto-fill
Security	Rate limiting, HTTPS enforcement, secret management, stricter input validation
Compliance	DPDP Act alignment, encryption at rest, explicit user consent flows
Roles	Differentiate caregiver vs. patient permissions (e.g. caregiver managing multiple patients)

11. Deployment & How to Run

11.1 Live Deployment

Frontend: <https://ai-care-navigator.vercel.app>

Backend API: <https://ai-care-navigator.onrender.com/api/health>

Source code: <https://github.com/Shivamkumar31/AI-Care-navigator>

11.2 Local Setup (Docker — recommended)

git clone the repository, then from the project root run: `docker-compose up --build`. This starts MongoDB, the backend (port 5000), and the frontend (port 3000). Seed the hospital dataset once with: `docker exec -it acn-backend node seed/seed.js`

11.3 Manual Setup

- Backend: `cd backend && npm install && npm run seed && npm run dev`
- Frontend: `cd frontend && npm install && npm run dev`
- Environment variables are documented in `backend/.env.example` and `frontend/.env.local.example`

12. Judging Criteria Alignment

Criterion	Weight	How this submission addresses it
Vision & pathway	20%	Unifies fragmented insurance, hospital, and care-journey information into one India-specific flow, with a clear path from MVP to production (Section 10).
Working solution	30%	Fully functional, deployed, end-to-end tested: registration, policy entry, hospital matching, and journey tracking all work against a live database.
Innovative AI use	30%	A transparent, rule-based decision-support engine that is fully explainable and auditable — an important property in insurance and healthcare contexts, where opaque recommendations are a liability.
Enterprise	20%	A realistic, multi-scheme dataset spanning six cities, explicit non-diagnostic and non-binding disclaimers throughout, and documented production-readiness roadmap.

13. Conclusion

AI Care Navigator demonstrates a working, deployed, production-architected solution to a real and widespread problem: the disconnect between insurance entitlements and hospital care in India. It is built on a genuine three-tier stack, tested end to end, and designed so that the single synthetic component — the hospital dataset — can be replaced with real data without any change to the application's architecture or logic. The project is submitted with full source code, this documentation, a live working demo, and an accompanying presentation and demo video.

— Team rock